# FingerNib

## Summary

FingerNib is (or will be) a cross-platform program that allows the user to write text, including but not limited to letters, numbers, punctuation, other common symbols such as @ or $, and math symbols such as $\in$, $\sum$, and $\int$. While running, the program waits invisibly in the background. Once the user presses their chosen hotkey, FingerNib mode is enabled. Their track pad will stop moving the mouse and the gestures will instead be collected as (x,y,t) stroke data. This data is passed to a neural network written in Rust, which determines what character was typed. Previously typed characters are then used for disambiguation, and the character is typed. Because the program outputs characters as though they were typed, FingerNib is completely agnostic of the app that the user wants to type in. They are free to use Obsidian, Overleaf, Google Docs, or even nano. Once the user is done writing, they can hit their hotkey again and FingerNib returns to the background.

FingerNib also comes with a GUI for configuration utilizing Tauri. With this, the user can configure numerous aspects of the app, including but not limited to:

1. The enable hotkey
2. Which characters are enabled or disabled
3. What a character expands to, i.e. does writing '$\in$' output "\in" or "in"?
4. Define multiple contexts which have different enabled and disabled keys, i.e. one for LaTex and one for Typst

## Motivation

According to numerous studies handwriting has significant benefits to learning compared to typing. However, there are undeniable benefits to typing. Typed notes are much more versatile. They can be easily backed up, searched through, copied, and pasted. They can also be easily converted to different forms or otherwise enhanced, like creating flashcards or wiki links. An additional weakness of typing is writing math. While apps with latex support or built-in equations like obsidian or google docs allow for equations to be nicely formatted, it takes much longer to type `\frac{n!}{(n-k!)(k!)}` than it would take to write the corresponding fraction by hand. The goal of this project is to combine the benefits of handwriting with the benefits of typing.

## Implementation

The overarching projects is composed of three main components. The first step is training the model, which is done in Python using PyTorch. The model is trained on the Handwriting Recognition Toolkit (HWRT) dataset, which contains 369 different letters, numbers and symbols: A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z

a, b, c, d, e, f, g, h, i, j, k , l , m , n , o , p , q , r , s , u , v , w , x , y, z
0, 1, 2, 3, 4,5, 6, 7, 8, 9
$\pi$ $\alpha$, $\beta$ , $\sum$ , $\sigma$, $\Sigma$, $\gamma$ , $\Gamma$ , $\delta$ , $\Delta$ , $\zeta$ , $\eta$ , $\theta$ , $\Theta$ , $\epsilon$ , $\varepsilon$ , $\iota$ , $\kappa$ , $\varkappa$ , $\lambda$ , $\Lambda$ , $\mu$ ,
$\nu$ , $\xi$ , $\Xi$ , $\Pi$, $\rho$ , $\varrho$ , $\tau$ , $\phi$ , $\Phi$ , $\varphi$ , $\chi$ , $\psi$ , $\Psi$ , $\omega$ , $\Omega$ , $\partial$ , $\int$ , $\cdot$ , $\leq$ , $\geq$ , $<$ , $>$ ,
$\subset$ , $\supset$ , $\subseteq$ , $\supseteq$ , $\cong$ , $\propto$ , $-$ , $+$ , $\mathbb{R}$ , $\$$ , { , ©, . . . , } , § † £ , & , # , % , ✓ , ®
§ , $\amalg$ , $\cup$ , $\oplus$ , $\times$ , $*$ , $\triangleleft$ , $\otimes$ , $\triangleright$ , $\diamond$ , $\pm$ , $\div$ , $\bullet$ , $\setminus$ , $\uplus$ , $\cap$ , $\mp$ , $\sqcap$ , $\vee$ , $\odot$
$\sqcup$ , $\wedge$ , $\circ$ , $\ominus$ , $\star$ , $\wr$ , $\overline{\wedge}$ , $\circledcirc$ , $\boxdot$ , $\ltimes$ , $\boxplus$ , $\boxtimes$ , $\rtimes$ , $\circledast$ , $\triangleleft$ , $\prod$ , $\coprod$, $\oint$, $\aleph$,
$\iiint$ , $\approx$ , $\equiv$ , $\not\equiv$ , $\perp$ , $\asymp$ , $\frown$ , $\prec$ , $\succ$ , $\bowtie$ , $\preceq$ , $\succeq$ , $\mid$ , $\vdash$ , $\dashv$ , $\models$ , $\sim$ , $\doteq$
$\parallel$ , $\simeq$ , $\backsim$ , $\multimap$ , $\pitchfork$ , $\therefore$ , $\because$ , $\between$ , $\preccurlyeq$ , $\propto$ , $\Vdash$ , $\models$ , $\nmid$ , $\not\models$ , $\sqsubset$ , $\not\subseteq$ , $\subsetneq$ , $\subsetneq$ , $\neq$ $\to$
$\geqslant$ , $\gtrless$ , $\lesssim$ , $\gtrsim$ , $\leqslant$ , $\trianglelefteq$ , $\blacktriangleright$ , $\triangleq$ , $\Downarrow$ , $\downarrow$ , $\Rightarrow$ , $\hookrightarrow$ , $\Longleftrightarrow$ , $\searrow$ , $\longmapsto$ , $\leftarrow$
$\Longrightarrow$ , $\uparrow$ , $\Leftarrow$ , $\longrightarrow$ , $\Leftrightarrow$ , $\mapsto$ , $\leftrightarrow$ , $\nearrow$ , $\rightleftharpoons$ , $\rightharpoonup$ , $\rightsquigarrow$ , $\circlearrowleft$ , $\rightleftarrows$ , $\circlearrowright$ , $\rightrightarrows$ , $\rightsquigarrow$ , $\curvearrowright$ , $\twoheadrightarrow$ , $\Rightarrow$
$\nrightarrow$ , $\restriction$, $\vartheta$ , $\varpi$ , $\perp$ , $\forall$ , $\ni$ , $\top$ , $\ell$ , $\hbar$
$\in$ , $\notin$ , $\wp$ , $\exists$ , $\Im$ , $\Re$ , $\nexists$ , $\langle$ , $\rangle$ , $\lceil$ , $\rceil$ , $\lfloor$ , $\rfloor$ [ ] | | / ,
$\vdots$ , $\ddots$ , . . . , $\aleph$ , $\infty$ , $\prime$ , $\angle$ , $\diamondsuit$ , $\sharp$ , $\setminus$ , $\emptyset$ , $\nabla$ , $\flat$ , $\clubsuit$ , $\heartsuit$ , $\neg$ , $\triangle$ , $\surd$ , $\triangleleft$ , $\square$ , $\triangledown$
■ , $\diamondsuit$ , $\varnothing$ , $\triangle$ , $\iddots$ , $\mathcal{A}$ , $\mathcal{B}$ , $\mathcal{C}$ , $\mathcal{D}$ , $\mathcal{E}$ , $\mathcal{F}$ , $\mathcal{G}$ , $\mathcal{H}$ , $\mathcal{L}$ , $\mathcal{M}$ , $\mathcal{N}$ , $\mathcal{O}$ , $\mathcal{P}$ , $\mathcal{R}$ , $\mathcal{S}$ , $\mathcal{T}$
$\mathcal{U}$ , $\mathcal{X}$ , $\mathcal{Z}$ , $\mathfrak{A}$ , $\mathfrak{M}$ , $\mathfrak{S}$ , $\mathfrak{X}$ , $\mathbb{1}$ , $\mathbb{C}$ , $^\circ$ , $\Omega$ , $\mathbb{Q}$ , $\mathbb{N}$ , $\mathbb{Z}$ , $\mathbb{H}$ , $\mathring{A}$
There are actually even more than this in the dataset. Some of them don't render properly with MathJax *or* LaTex, so I have excluded them from the list. Clearly, they will not be missed. Most of these symbols are superfluous, but since the user can disable characters as they please with the configuration GUI, this won't really hurt anything, and if the user truly needs to write an inverted ampersand, they are free to do so. There are, however, a few issues with this dataset. Consider, for example, $\sum$ (\sum) and $\Sigma$ (\Sigma). Is it truly possible to tell when a person writes the first versus the second? Not likely.

```
epoch  1  train loss 1.3718 acc 68.1%  |  val loss 0.8314 acc 78.1%
epoch  2  train loss 0.6964 acc 80.2%  |  val loss 0.7375 acc 79.0%
epoch  3  train loss 0.5979 acc 82.2%  |  val loss 0.6772 acc 80.4%
epoch  4  train loss 0.5431 acc 83.3%  |  val loss 0.6506 acc 81.7%
epoch  5  train loss 0.5033 acc 84.1%  |  val loss 0.6443 acc 81.0%
epoch  6  train loss 0.4705 acc 85.0%  |  val loss 0.6507 acc 81.3%
epoch  7  train loss 0.4398 acc 85.5%  |  val loss 0.6424 acc 81.3%
epoch  8  train loss 0.4130 acc 86.3%  |  val loss 0.6434 acc 81.1%
epoch  9  train loss 0.3832 acc 86.9%  |  val loss 0.6490 acc 81.4%
epoch 10  train loss 0.3557 acc 87.7%  |  val loss 0.6699 acc 81.2%
```

When training the model, we see that the model stops improving on the validation data as early as epoch 4. Unfortunately, an accuracy of 80% is simply not acceptable - missing one in every five characters would be a terrible user experience. Ambiguity between symbols likely takes a share of the blame.

```
top-1: 81.7%
top-2: 92.8%
top-3: 95.6%
top-4: 96.7%
top-5: 97.2%
 144  \Sigma  ->  \sum
  68  \bot  ->  \perp
  39  \propto  ->  \alpha
  33  \vDash  ->  \models
  32  \triangle  ->  \Delta
  32  \Pi  ->  \pi
  30  |  ->  \mid
  28  \alpha  ->  \propto
  27  \shortrightarrow  ->  \rightarrow
  27  \mathds{1}  ->  \mathbb{1}
  27  \backslash  ->  \setminus
  27  \parallel  ->  \|
  26  \circ  ->  \degree
  26  \prod  ->  \pi
  23  \degree  ->  \circ
  23  \varnothing  ->  \emptyset
  23  \epsilon  ->  \in
  21  \O  ->  \emptyset
  19  \Longrightarrow  ->  \Rightarrow
  19  \checked  ->  \checkmark
  18  \Pi  ->  \prod
  18  \mathcal{E}  ->  \varepsilon
  17  \Lambda  ->  \wedge
  17  \rightsquigarrow  ->  \leadsto
  17  \ohm  ->  \Omega
```

When considering the second best guess of the model, accuracy increases by more than 10%, indicating that there are a large number of incorrect answers which have very strong second guesses, i.e. difficult to distinguish symbols. When looking at the actual errors, we see that this is generally the case. $\Sigma$ (\Sigma) is mistakenly identified as $\sum$ (\sum) 144 times. Other mixups include $\perp$ (\bot) and $\perp$ (\perp), \shortrightarrow and \rightarrow, and $\Omega$ (\ohm) and $\Omega$ (\Omega) . For these situations where two symbols are truly indistinguishable, we can simply remove one of the two offending symbols. The argument could be made that this is not actually acceptable: if we remove \ohm in favor of \Omega, someone could need \ohm and only have \Omega. In reality, this is a non-issue. Using the configuration GUI, the user can decide what writing $\Omega$ should output. If they want \Ohm, they can have \Ohm. If they want \Omega, they can have \Omega. If they really wanted, they could make it output $S = k_B \ln \Omega$ or something unrelated. The only thing they cannot have is an $\Omega$ that becomes two different things arbitrarily.

Some of the errors are still issues that need to be ironed out of the network. $\epsilon$ and $\in$, for example, are two similar looking but distinct symbols which should not be confused. These are problems that will be worked out as the strategy for training the model becomes less naive and more accurate. The goal is to have a roughly 95% overall accuracy for the final product, with a near perfect accuracy for the most common symbols, namely the Latin alphabet and digits 0-9.

In order to resolve issues with similar looking characters, the models output will likely be combined with the result of an n-gram language model, using the context of the character to help decide what should be used. Additionally, if the user does come across an error while writing, they will have the ability to cycle through the model's next best guesses, allowing them to quickly move on from the issue with minimal interruption.

Once the network is trained it will be exported to ONNX and used in the second part of FingerNib. The model will run in Rust via the Burn framework, which is faster, lighter, and supports more GPUs than PyTorch. This is important, since the final product will likely be running multiple networks and should be performant on as many devices as possible. The Rust program will also handle most of the other tasks, such as enabling writing mode, taking trackpad input, and outputting keys.

The third component of FingerNib is a configuration GUI, which allows the user to configure virtually any aspect of their experience. This will be a react app in Tauri, for a sleek cross-platform experience. The user has many options to customize FingerNib to their liking, including but not limited to:

- Enabling and disabling characters
- Changing keybinds and shortcuts
- Adding biases to common characters
- Changing what a symbol outputs
    - A symbol's output can include *capture groups*. Capture groups allow for content to be inserted in another character's expansion. This is useful for things like $\int$ (\int). The user can set the integral symbol's expansion to be "\int*{(capture 1)}^{(capture 2)}". When the user writes the symbol, FingerNib types until it reaches the first capture group (in this case, "\int{" ). It then allows the user to write as normal, until they press a button to break out of the current capture group and move on to the next one. When they move to the next capture group, FingerNib types all of the characters up to that point, and this continues until all of the capture groups are completed. With this feature, the user can conveniently write complex math formulas such as $\int*{a}^{b}f(x)dx$ (\int_{a}^{b}f(x)dx) without needing to worry about writing out clunky symbols like '{'.
- Creating different contexts
    - If a user uses both LaTex and Typst, they would find that they would want to have $\in$ expand to "\in" when using LaTex, but "in" when using Typst. To solve this problem, they

can create a LaTex context and a Typst context (which will both be available as defaults). Each context has its own settings, so the user can easily switch to whichever setup they need.